CASE STUDY ASSIGNMENT

Academic Year: 2026–2027

Student Name:	
USN:	
Semester / Section:	
Subject Name & Code:	Theory of Computations - 24CS52
Assigned Set:	SET-A / SET-B / SET-C / SET-D / SET-E
Case Study Title:	Web Content Analyzer: Pattern Extraction & Lexical Analysis (RE vs DFA)

ASSESSMENT AND MARKS EVALUATION

Maximum Marks: 50

Sl. No.	Assessment Criteria	Maximum Marks	Marks Awarded
1	Problem understanding and identification of patterns/requirements	5	
2	Development of Regular Expressions for the identified patterns (minimum 5)	10	
3	Conversion of selected RE into DFA with correct state-transition diagram and transition table	10	
4	Implementation using Python/C (RE and DFA)	10	
5	Testing, validation and analysis of results	7	
6	Comparison, interpretation and discussion of limitations (RE vs DFA)	3	
7	Documentation, output screenshots and presentation/viva	5	
TOTAL		**50**	

Course Instructor Remarks: _____

Date of Evaluation: _____

Faculty Signature: _____

Student Signature: _____

PROBLEM STATEMENT:

In formal language theory and compiler design, regular languages (L_3 in the Chomsky Hierarchy) constitute the foundational formal model for lexical analysis, tokenization, and pattern matching. Real-world web documents (such as academic portals, news publishers, and technical discussion platforms) contain vast volumes of semi-structured text embedded with vital syntactic entities, including email addresses, telephone contacts, calendar dates, hyperlinks, social taxonomy tags, timestamps, and specific grammatical inflections.

The objective of this case study is to design, implement, empirically evaluate, and benchmark a full-stack **Web Content Analyzer** that extracts structured patterns from real-world web pages by contrasting two fundamental theoretical paradigms:

1. **Declarative Regular Expressions (RE):** High-level pattern specifications executed via Python's standard regular expression engine (`re` module), which operates through an automated backtracking matching algorithm.

2. **Imperative Hand-Built Deterministic Finite Automaton (DFA):** A transition-table-driven finite state automaton designed and coded from first mathematical principles without employing any regular expression matching libraries.

Core Objectives and Project Scope

- **Domain Ingestion:** Ingest, sanitize, and analyze heterogeneous web documents representing three distinct domains: an Educational Regulatory Portal (University Grants Commission), a National Breaking News Portal (The Indian Express), and an active Software Engineering Community (DEV Community).
- **Formal Pattern Identification:** Formulate and specify mathematically rigorous regular expressions for at least five distinct token languages commonly encountered in web extraction pipelines.
- **DFA Formal Derivation:** Select one identified regular language—specifically English words ending in the suffix `"-ing"` with at least one preceding letter ($[A-Za-z]^+ing$)—and formally derive its equivalent minimized DFA through Thompson's NFA construction, subset (powerset) construction, and Hopcroft's state minimization algorithm. Formulate the complete formal 5-tuple $M = (Q, \Sigma, \delta, q_0, F)$, state-transition diagram, and transition table.
- **Dual Implementation:** Construct modular Python implementations of both the declarative RE engine and the table-driven DFA engine.
- **Empirical Validation & Benchmarking:** Verify 100% bit-for-bit extraction equivalence between RE and DFA on real web text, execute 500-iteration microsecond timing benchmarks across all pages and an extended 1 MB corpus, evaluate lines of code (LOC), auxiliary space complexity, and assess resistance to Regular Expression Denial of Service (ReDoS).
- **Interactive Cloud Prototype:** Build a modern, responsive single-page user interface and deploy the application to cloud serverless infrastructure (Vercel) for live public verification.

DATASET / INPUT DOCUMENT

To ensure the empirical investigation evaluates real-world linguistic diversity, noisy document structures, and varied token densities, three publicly accessible web pages were collected and preprocessed into clean text corpora. In addition, an extended 1 MB synthetic scaling corpus was compiled for asymptotic stress testing.

1. Web Page Corpus Overview

Snapshot ID	Domain Category	Source Title	Live Source URL	Character Count	Line Count	Total Extracted Tokens
`page1`	Educational Regulatory Portal	University Grants Commission (UGC)	https://www.ugc.gov.in/	27,264	968	238
`page2`	National News Portal	The Indian Express (Helpline Report)	[https://indianexpress.com/article/health-wellness/...](https://indianexpress.com/)	31,938	783	388
`page3`	Tech Blog & Developer Community	DEV Community (Python Index)	https://dev.to/t/python	18,835	648	241
`synthetic_1mb`	Scaled Benchmark Corpus	Aggregated Multi-Domain Corpus	Internal Benchmark Suite	1,024,000	28,450	3,456

2. Preprocessing & Sanitization Pipeline

Web documents in raw HTML format contain extensive DOM boilerplate, client-side JavaScript, CSS style declarations, SVG graphics, and metadata headers that do not represent textual content. The collection script (`scripts/collect_pages.py`) executed the following cleaning stages:

1. **SSRF-Protected HTTP Fetching:** Implemented in `core/scraper.py`, the fetcher validates target DNS hostnames and strictly prohibits loopback addresses (`127.0.0.1`, `::1`), private subnets (`10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`), and non-HTTP protocols, enforcing a 10-second socket timeout and a 2 MB size cap.

2. **DOM Noise Stripping:** Removed `<script>`, `<style>`, `<noscript>`, `<svg>`, `<header>`, `<footer>`, and `<nav>` tags using BeautifulSoup parser tree operations.

3. **Hyperlink Token Preservation:** Converted anchor tags of the form `<a href="URL">ANCHOR_TEXT</a>` into explicit text annotations `ANCHOR_TEXT (URL)` prior to tag stripping, ensuring that hyperlinks and mailto references remain accessible for lexical analysis.

4. **Whitespace Normalization:** Collapsed repeated blank lines while preserving single newline characters to facilitate accurate line and column coordinate mapping.

3. Domain Characteristics & Token Densities

- **Page 1 (UGC - Educational Portal):** Characterized by official regulatory notices, statutory guidelines, committee circulars, government `.gov.in` and `.ac.in` contact emails, formal DD-MM-YYYY dates, and administrative vocabulary (e.g., *"Monitoring"*, *"Following"*, *"Training"*).
- **Page 2 (The Indian Express - News Portal):** Characterized by journalism reportage, emergency suicide prevention helplines, verified 10-digit Indian mobile numbers (`+91 98...`), ISO timestamps, internal hyperlinks, and editorial participles.
- **Page 3 (DEV Community - Tech Blog):** Dominated by programming discussions, code identifiers, developer links, open-source repositories, technical vocabulary, and heavy usage of social metadata tags (`#python`, `#webdev`, `#opensource`, `#ai`).

PATTERN IDENTIFICATION AND REGULAR EXPRESSIONS

A minimum of five patterns were required for the case study. Seven distinct formal regular language token patterns were identified, specified, and implemented. Each pattern corresponds to a regular language $L \subseteq \Sigma^*$ definable over ASCII/Unicode character sets.

Sl. No.	Pattern Key	Formal Token Language	Regular Expression Specification	Flags
1	`email`	Standard Email Address	`[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`	None
2	`mobile`	Indian Mobile Number	`(?<!\d)(?:(?:\+91\	91\
3	`date`	Calendar Date (DD-MM-YYYY)	`\b(0[1-9]\	[12]\d\
4	`url`	Web Hyperlink (HTTP/HTTPS)	`https?://[^\s<>\x22\'\)]*[^\s<>\x22\'\).,;:?!]`	None
5	`hashtag`	Social Tag / Category	`(?<![\w#])#[A-Za-z][A-Za-z0-9_]*`	None
6	`time`	24-Hour Clock Time (HH:MM)	`\b([01]\d\	2[0-3]):[0-5]\d\b`
7	`ing_word`	English Words Ending in '-ing'	`\b[A-Za-z]+ing\b`	`re.IGNORECASE`

Detailed Technical Breakdown of Identified Patterns

1. Standard Email Address (`email`)

- **Regular Expression:** `[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`
- **Syntactic Explanation:**
- `[A-Za-z0-9._%+-]+`: Matches the local part (mailbox identifier) consisting of one or more alphanumeric characters or sanctioned symbols (`.`, `_`, `%`, `+`, `-`).
- `@`: The literal delimiter separating the local mailbox identifier from the fully qualified domain name.
- `[A-Za-z0-9.-]+`: Matches domain host labels separated by periods or hyphens.
- `\.`: Literal period preceding the Top-Level Domain (TLD).
- `[A-Za-z]{2,}`: Requires an alphabetic TLD with a minimum length of 2 characters (e.g., `.in`, `.gov`, `.education`), eliminating single-letter errors.
- **Positive Valid Tokens:** `covid19help.ugc@gov.in`, `netjrf-ugc@gov.in`, `helpdesk@nsp.gov.in`.
- **Negative Rejected Inputs:** `user@domain`, `@missinguser.com`, `user@.com`.

2. Indian Mobile Number (`mobile`)

- **Regular Expression:** `(?<!\d)(?:(?:\+91|91|0)[\s-]?)?[6-9]\d{9}(?!\d)`
- **Syntactic Explanation:**
- `(?<!\d)`: Negative lookbehind assertion ensuring the token is not preceded by a decimal digit, preventing substring extraction from longer numerical sequences (e.g., student IDs, bank accounts).
- `(?:(?:\+91|91|0)[\s-]?)?`: Non-capturing group matching optional Indian dialing prefixes: `+91`, `91`, or `0`, followed by an optional separator space or hyphen.
- `[6-9]`: Mandates that the 10-digit subscriber number starts with 6, 7, 8, or 9 in compliance with the National Numbering Plan administered by the Telecom Regulatory Authority of India (TRAI).
- `\d{9}`: Exactly nine subsequent decimal digits, ensuring a 10-digit national number.
- `(?!\d)`: Negative lookahead assertion preventing partial matches in numbers of 11 or more digits.
- **Positive Valid Tokens:** `+91 9820466726`, `9876543210`, `098204-66726`.
- **Negative Rejected Inputs:** `1234567890` (invalid initial digit `1`), `98765` (too short), `98765432101` (too long).

3. Calendar Date (DD-MM-YYYY) (`date`)

- **Regular Expression:** `\b(0[1-9]|[12]\d|3[01])-(0[1-9]|1[0-2])-(19|20)\d{2}\b`
- **Syntactic Explanation:**

- `\b`: Word boundary anchor asserting a transition between a word character and non-word character.
- `(0[1-9]|[12]\d|3[01])`: Day component restricting valid days to `01` through `31`.
- `-`: Literal hyphen delimiter.
- `(0[1-9]|1[0-2])`: Month component restricting valid months to `01` through `12`.
- `-`: Literal hyphen delimiter.
- `(19|20)\d{2}`: Year component restricted to centuries 1900–2099, preventing illogical dates like `01-01-0000`.
- `\b`: Closing word boundary anchor.
- **Positive Valid Tokens:** `01-07-2024`, `15-08-1947`, `22-12-2003`.
- **Negative Rejected Inputs:** `32-01-2024` (day exceeds 31), `15-13-2024` (month exceeds 12), `00-05-2020`.

4. Web Hyperlink (HTTP/HTTPS) (`url`)

- **Regular Expression:** `https?://[^\s<>\x22\'\)]*[^\s<>\x22\'\).,;:?!]`
- **Syntactic Explanation:**
- `https?://`: Matches the transfer protocol scheme (`http://` or `https://`).
- `[^\s<>\x22\'\)]*`: Greedily consumes valid URL characters while stopping at whitespace, angle brackets, quotes, or closing parentheses.
- `[^\s<>\x22\'\).,;:?!]`: Ensures that trailing punctuation characters (`.`, `,`, `;`, `:`, `?`, `!`) often found at the end of English sentences are cleanly stripped and not considered part of the URL.
- **Positive Valid Tokens:** `https://www.ugc.gov.in/notices`, `https://dev.to/t/python`.
- **Negative Rejected Inputs:** `ftp://files.example.com`, `http://`.

5. Social Hashtag (`hashtag`)

- **Regular Expression:** `(?<![\w#])#[A-Za-z][A-Za-z0-9_]*`
- **Syntactic Explanation:**
- `(?<![\w#])`: Negative lookbehind preventing matching inside URL fragment identifiers or compound symbols.
- `#`: Mandatory literal hash symbol.
- `[A-Za-z]`: Enforces that the tag must begin with an alphabetic character, preventing pure numeric tags like `#123`.
- `[A-Za-z0-9_]*`: Consumes subsequent alphanumeric characters or underscores.
- **Positive Valid Tokens:** `#python`, `#webdev`, `#opensource`, `#machinelearning`.
- **Negative Rejected Inputs:** `#123`, `tag#python`, `##doublehash`.

6. 24-Hour Clock Time (`time`)

- **Regular Expression:** `\b([01]\d|2[0-3]):[0-5]\d\b`
- **Syntactic Explanation:**
- `\b`: Word boundary anchor.
- `([01]\d|2[0-3])`: Restricts hours to the range `00` through `23`.
- `:`: Literal colon separator.
- `[0-5]\d`: Restricts minutes to the range `00` through `59`.
- `\b`: Closing word boundary anchor.
- **Positive Valid Tokens:** `09:30`, `14:15`, `23:59`, `00:00`.

- **Negative Rejected Inputs:** `24:00` (hours cannot be 24), `12:60` (minutes exceed 59).

7. English Words Ending in '-ing' (`ing_word`)

- **Regular Expression:** `\b[A-Za-z]+ing\b` (compiled with `re.IGNORECASE`)
- **Syntactic Explanation:**
- `\b`: Word boundary ensuring the token is an independent lexical word.
- `[A-Za-z]+`: One or more alphabetic prefix letters before the suffix `"ing"`. This enforces that the token length must be at least 4 letters, correctly rejecting the 3-letter word `"ing"`.
- `ing`: The literal suffix substring.
- `\b`: Closing word boundary.
- **Positive Valid Tokens:** `learning`, `singing`, `monitoring`, `violating`, `ring`.
- **Negative Rejected Inputs:** `ing` (length 3, 0 prefix letters), `springboard` (suffix is `board`), `123ing` (preceded by digit).

RE IMPLEMENTATION USING PYTHON

1. Implementation Approach & Architecture

The Regular Expression engine is designed using a modular, decoupled architecture consisting of:

1. **Formal Pattern Registry (`core/patterns.py`):** Encapsulates the regex strings, human-readable labels, descriptive summaries, and cached compiled regex objects (`re.Pattern`) using `re.compile()`. Compiling regular expressions once at initialization amortizes the compilation overhead and enables fast multi-pass extraction.

2. **Extraction Engine (`core/re_engine.py`):** Operates on clean text documents using `finditer()`, which returns iterator match objects without allocating a monolithic list of substrings. For each detected token, the engine computes:

- 0-based character start and end offsets.
- 1-based (line, column) human-readable coordinates by performing a fast binary search (`bisect`) over precomputed newline offsets (`core/utils.py`).

3. **Microsecond Benchmarking (`benchmark()`):** Utilizes `time.perf_counter()` to record high-resolution CPU execution timings across repeated iterations.

---

2. Complete Python Source Code: Pattern Registry (`core/patterns.py`)

```
"""Pattern Registry for Web Content Analyzer (RE vs DFA).

Defines regular expressions for tokenizing and extracting structured patterns
from web text: Email, Indian Mobile Numbers, Dates, URLs, Hashtags, 24-hr Time,
and '-ing' words.
"""

import re
from typing import Any

# ============================================================================
# PATTERN SPECIFICATIONS AND REGULAR EXPRESSIONS
# ============================================================================

# 1. EMAIL PATTERN
# Regex: [A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}
# Explanation:
# - [A-Za-z0-9._%+-]+ : Matches the local part (username) composed of alphanumeric
#   characters or standard email special characters (. _ % + -).
# - @                 : Literal '@' separator between local part and domain.
# - [A-Za-z0-9.-]+    : Matches domain name labels separated by dots or hyphens.
# - \.                : Literal dot separating domain and top-level domain (TLD).
# - [A-Za-z]{2,}      : Matches the TLD consisting of at least 2 alphabetic characters.
EMAIL_REGEX = r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}"
```

```
# 2. INDIAN MOBILE NUMBER PATTERN
# Regex: (?<!\d)(?:(?:\+91|91|0)[\s-]?)?[6-9]\d{9}(?!\d)
# Explanation:
# - (?<!\d)           : Negative lookbehind asserting the preceding character is not
#                       a digit (prevents partial match in longer number sequences).
# - (?:(?:\+91|91|0)[\s-]?)? : Optional country/trunk code: '+91', '91', or '0',
#                       optionally followed by a single whitespace or hyphen.
# - [6-9]             : Indian mobile numbers mandate starting with digit 6, 7, 8, or 9.
# - \d{9}             : Followed by exactly 9 more decimal digits (total 10 digits).
# - (?!\d)            : Negative lookahead asserting the following character is not
#                       a digit (rejects 11+ digit numbers).
MOBILE_REGEX = r"(?<!\d)(?:(?:\+91|91|0)[\s-]?)?[6-9]\d{9}(?!\d)"

# 3. DATE PATTERN (DD-MM-YYYY)
# Regex: \b(0[1-9]|[12]\d|3[01])-(0[1-9]|1[0-2])-(19|20)\d{2}\b
# Explanation:
# - \b                : Word boundary ensuring the date stands alone.
# - (0[1-9]|[12]\d|3[01]) : Day group strictly matching 01-09, 10-29, 30, or 31.
# - -                 : Literal hyphen separator.
# - (0[1-9]|1[0-2])   : Month group strictly matching 01-09, 10, 11, or 12.
# - -                 : Literal hyphen separator.
# - (19|20)\d{2}      : Year group restricted to centuries 1900-2099.
# - \b                : Word boundary.
DATE_REGEX = r"\b(0[1-9]|[12]\d|3[01])-(0[1-9]|1[0-2])-(19|20)\d{2}\b"

# 4. URL PATTERN
# Regex: https?://[^\s<>\x22\'\)]*[^\s<>\x22\'\).,;:?!]
# Refinement from base https?://[^\s<>"')]+:
# Uses [^\s<>\x22\'\)]* followed by terminal [^\s<>\x22\'\).,;:?!] to cleanly exclude
# sentence-ending trailing punctuation (such as '.', ',', ';', ':', '?', '!') and enclosing brackets,
# ensuring valid, clean URLs when embedded in prose or parenthetical links " (href)".
# Explanation:
# - https?://                 : Protocol 'http://' or 'https://'.
# - [^\s<>\x22\'\)]*          : Zero or more valid URL characters (excluding whitespace and delimiters).
# - [^\s<>\x22\'\).,;:?!]     : Terminal URL character excluding sentence punctuation.
URL_REGEX = r"https?://[^\s<>\x22\'\)]*[^\s<>\x22\'\).,;:?!]"

# 5. HASHTAG PATTERN
# Regex: (?<![\w#])#[A-Za-z][A-Za-z0-9_]*
# Refinement from base (?<!\w)#[A-Za-z][A-Za-z0-9_]*:
# Uses negative lookbehind (?<![\w#]) to prevent matching consecutive hashes
# such as Markdown headings (e.g. ## Heading) or double hashes (##tag).
# Explanation:
# - (?<![\w#])       : Negative lookbehind asserting not preceded by a word char or another '#'.
# - #                : Literal hash symbol '#'.
# - [A-Za-z]         : Initial tag character must be an alphabetic letter (rules out #123).
# - [A-Za-z0-9_]*    : Subsequent characters can be alphanumeric or underscores.
HASHTAG_REGEX = r"(?<![\w#])#[A-Za-z][A-Za-z0-9_]*"

# 6. TIME PATTERN (HH:MM 24-HOUR)
# Regex: \b([01]\d|2[0-3]):[0-5]\d\b
# Explanation:
# - \b                : Word boundary.
# - ([01]\d|2[0-3])   : Hours group matching 00-19 or 20-23 (valid 24-hour range).
# - :                 : Literal colon delimiter.
# - [0-5]\d           : Minutes group matching 00-59.
# - \b                : Word boundary.
TIME_REGEX = r"\b([01]\d|2[0-3]):[0-5]\d\b"

# 7. ING-WORD PATTERN
# Regex: \b[A-Za-z]+ing\b
# Explanation:
# - \b                : Word boundary.
# - [A-Za-z]+         : At least one alphabetic character before 'ing' (ensures 'ing' alone
#                       with 0 preceding letters does not match, minimum 4 letters).
# - ing               : Literal suffix 'ing'.
# - \b                : Word boundary.
ING_WORD_REGEX = r"\b[A-Za-z]+ing\b"


# ================================================================================
# PATTERN REGISTRY DICTIONARY
# ================================================================================

PATTERNS: dict[str, dict[str, Any]] = {
    "email": {
        "key": "email",
        "label": "Email Address",
        "regex": EMAIL_REGEX,
        "flags": 0,
        "description": "Standard email address matching local username, '@', domain, and a 2+ letter TLD.",
        "example": "student.support@university.edu",
    },
```

```
    "mobile": {
        "key": "mobile",
        "label": "Indian Mobile Number",
        "regex": MOBILE_REGEX,
        "flags": 0,
        "description": "Indian 10-digit mobile number starting with 6-9, optional +91/91/0 prefix, space/hyphen separator.",
        "example": "+91 9876543210",
    },
    "date": {
        "key": "date",
        "label": "Date (DD-MM-YYYY)",
        "regex": DATE_REGEX,
        "flags": 0,
        "description": "Date in DD-MM-YYYY format with valid ranges (DD: 01-31, MM: 01-12, YYYY: 1900-2099).",
        "example": "15-08-1947",
    },
    "url": {
        "key": "url",
        "label": "Web URL",
        "regex": URL_REGEX,
        "flags": 0,
        "description": "HTTP or HTTPS web link with trailing punctuation stripped cleanly.",
        "example": "https://www.example.org/admissions/index.html",
    },
    "hashtag": {
        "key": "hashtag",
        "label": "Hashtag",
        "regex": HASHTAG_REGEX,
        "flags": 0,
        "description": "Social/topic hashtag starting with '#' and a letter, followed by letters, digits, or underscores.",
        "example": "#TheoryOfComputation",
    },
    "time": {
        "key": "time",
        "label": "24-Hour Time (HH:MM)",
        "regex": TIME_REGEX,
        "flags": 0,
        "description": "24-hour clock time in HH:MM format (HH: 00-23, MM: 00-59).",
        "example": "14:30",
    },
    "ing_word": {
        "key": "ing_word",
        "label": "Words ending in 'ing'",
        "regex": ING_WORD_REGEX,
        "flags": re.IGNORECASE,
        "description": "English words ending in suffix 'ing' with at least 1 preceding letter (case-insensitive).",
        "example": "engineering",
    },
}


def get_pattern(key: str) -> dict[str, Any]:
    """Retrieve pattern metadata by key."""
    if key not in PATTERNS:
        raise KeyError(f"Pattern '{key}' not found in registry.")
    return PATTERNS[key]


def get_all_patterns() -> dict[str, dict[str, Any]]:
    """Return dictionary of all registered patterns suitable for JSON API response."""
    result = {}
    for key, p in PATTERNS.items():
        result[key] = {
            "key": p["key"],
            "label": p["label"],
            "regex": p["regex"],
            "flags": int(p["flags"]),
            "description": p["description"],
            "example": p["example"],
        }
    return result


def get_compiled_pattern(key: str) -> re.Pattern:
    """Return compiled re.Pattern instance for a given key."""
    p = get_pattern(key)
    return re.compile(p["regex"], p["flags"])
```

3. Complete Python Source Code: Extraction Engine (`core/re_engine.py`)

```
"""Regular Expression Pattern Extraction Engine.

Executes pattern matching against input text using Python's 're' module,
computes 0-based offsets and 1-based (line, column) positions, and collects
metrics (count, unique_count, execution time).
"""

import argparse
import json
import sys
import time
from pathlib import Path
from typing import Any

from core.patterns import PATTERNS, get_compiled_pattern, get_pattern
from core.utils import build_line_starts, offset_to_line_col


def extract_single_pattern(
    text: str, pattern_key: str, line_starts: list[int] | None = None
) -> dict[str, Any]:
    """Extract matches for a single pattern key from text.

    Args:
        text: Target text string.
        pattern_key: Key in PATTERNS registry.
        line_starts: Optional precomputed line start offsets.

    Returns:
        Dict with pattern metadata, count, unique_count, and list of matches.
    """
    if line_starts is None:
        line_starts = build_line_starts(text)

    pattern_meta = get_pattern(pattern_key)
    compiled_re = get_compiled_pattern(pattern_key)

    matches = []
    for match in compiled_re.finditer(text):
        m_text = match.group(0)
        start = match.start()
        end = match.end()
        line, col = offset_to_line_col(start, line_starts=line_starts)
        matches.append(
            {"text": m_text, "start": start, "end": end, "line": line, "column": col}
        )

    unique_items = {m["text"] for m in matches}

    return {
        "key": pattern_key,
        "label": pattern_meta["label"],
        "regex": pattern_meta["regex"],
        "count": len(matches),
        "unique_count": len(unique_items),
        "matches": matches,
    }


def extract(text: str, pattern_keys: list[str] | None = None) -> dict[str, Any]:
    """Extract all or selected patterns from text using Python regular expressions.

    Args:
        text: Input text string to analyze.
        pattern_keys: Optional list of pattern keys to run (defaults to all 7).

    Returns:
        Dict mapping pattern keys to their match results, plus aggregate stats
        and execution timing.
    """
    if pattern_keys is None:
        keys_to_run = list(PATTERNS.keys())
    else:
        keys_to_run = [k for k in pattern_keys if k in PATTERNS]

    start_time = time.perf_counter()
    line_starts = build_line_starts(text)

    results: dict[str, dict[str, Any]] = {}
    total_matches = 0
```

```
    for key in keys_to_run:
        pattern_result = extract_single_pattern(text, key, line_starts=line_starts)
        results[key] = pattern_result
        total_matches += pattern_result["count"]

    elapsed_seconds = time.perf_counter() - start_time
    elapsed_ms = round(elapsed_seconds * 1000, 3)

    return {
        "results": results,
        "patterns": results,  # Alias for flexibility
        "total_matches": total_matches,
        "elapsed_seconds": elapsed_seconds,
        "elapsed_ms": elapsed_ms,
    }


def benchmark(text: str, pattern_key: str, runs: int = 200) -> float:
    """Benchmark extraction time for a specific pattern over a number of runs.

    Args:
        text: Input text string.
        pattern_key: Key in PATTERNS registry.
        runs: Number of iterations (default 200).

    Returns:
        Average execution time per run in seconds.
    """
    runs = max(runs, 1)

    compiled_re = get_compiled_pattern(pattern_key)

    # Warm-up
    list(compiled_re.finditer(text))

    start_time = time.perf_counter()
    for _ in range(runs):
        list(compiled_re.finditer(text))
    total_duration = time.perf_counter() - start_time

    return total_duration / runs


def benchmark_all(text: str, runs: int = 200) -> dict[str, float]:
    """Benchmark all registered patterns and return average seconds per run."""
    timings = {}
    for key in PATTERNS.keys():
        timings[key] = benchmark(text, key, runs=runs)
    return timings


# ==============================================================================
# CLI INTERFACE: python -m core.re_engine --page 1
# ==============================================================================


def format_cli_table(results: dict[str, Any]) -> str:
    """Render a clean ASCII table of extraction results for terminal/screenshots."""
    headers = ["Pattern", "Count", "Extracted items", "Positions (L:C)"]
    col_widths = [24, 7, 36, 26]

    def truncate(s: str, max_len: int) -> str:
        s = s.replace("\n", " ")
        return s if len(s) <= max_len else s[: max_len - 3] + "..."

    sep = "+" + "+".join("-" * (w + 2) for w in col_widths) + "+"
    header_row = (
        "| " + " | ".join(h.ljust(col_widths[i]) for i, h in enumerate(headers)) + " |"
    )

    lines = [sep, header_row, sep]

    for key, data in results["results"].items():
        label = data["label"]
        count_str = str(data["count"])
        matches = data["matches"]

        if not matches:
            items_str = "(none)"
            pos_str = "-"
            lines.append(
                f"| {truncate(label, col_widths[0]).ljust(col_widths[0])} "
```

```
                f"| {count_str.rjust(col_widths[1])} "
                f"| {truncate(items_str, col_widths[2]).ljust(col_widths[2])} "
                f"| {truncate(pos_str, col_widths[3]).ljust(col_widths[3])} |"
            )
        else:
            # First row with count and first item
            first_m = matches[0]
            first_item = first_m["text"]
            first_pos = f"{first_m['line']}:{first_m['column']}"
            lines.append(
                f"| {truncate(label, col_widths[0]).ljust(col_widths[0])} "
                f"| {count_str.rjust(col_widths[1])} "
                f"| {truncate(first_item, col_widths[2]).ljust(col_widths[2])} "
                f"| {truncate(first_pos, col_widths[3]).ljust(col_widths[3])} |"
            )

            # Subsequent rows for up to 4 more items to keep table clean and informative
            max_extra = min(4, len(matches) - 1)
            for m in matches[1 : 1 + max_extra]:
                m_item = m["text"]
                m_pos = f"{m['line']}:{m['column']}"
                lines.append(
                    f"| {' '.ljust(col_widths[0])} "
                    f"| {' '.rjust(col_widths[1])} "
                    f"| {truncate(m_item, col_widths[2]).ljust(col_widths[2])} "
                    f"| {truncate(m_pos, col_widths[3]).ljust(col_widths[3])} |"
                )
            if len(matches) > 5:
                extra_cnt = len(matches) - 5
                lines.append(
                    f"| {' '.ljust(col_widths[0])} "
                    f"| {' '.rjust(col_widths[1])} "
                    f"| {f'(+{extra_cnt} more items)'.ljust(col_widths[2])} "
                    f"| {'...'.ljust(col_widths[3])} |"
                )

        lines.append(sep)

    summary_line = (
        f"Total Matches: {results['total_matches']} | "
        f"Extraction Time: {results['elapsed_ms']} ms"
    )
    return "\n".join(lines) + "\n" + summary_line


def main() -> None:
    parser = argparse.ArgumentParser(
        description="RE Engine: Extract TOC patterns from text."
    )
    parser.add_argument(
        "--page", type=int, choices=[1, 2, 3], help="Page snapshot number (1, 2, or 3)"
    )
    parser.add_argument("--file", type=str, help="Path to custom text file to analyze")
    parser.add_argument(
        "--benchmark",
        action="store_true",
        help="Run 200-iteration benchmark per pattern",
    )
    parser.add_argument("--json", action="store_true", help="Output raw JSON results")

    args = parser.parse_args()

    project_root = Path(__file__).resolve().parent.parent

    if args.page:
        page_file = project_root / "data" / "pages" / f"page{args.page}.txt"
        if not page_file.exists():
            print(f"Error: Page file not found: {page_file}", file=sys.stderr)
            sys.exit(1)
        text = page_file.read_text(encoding="utf-8")
        target_name = f"Page {args.page} ({page_file.name})"
    elif args.file:
        file_path = Path(args.file)
        if not file_path.exists():
            print(f"Error: File not found: {file_path}", file=sys.stderr)
            sys.exit(1)
        text = file_path.read_text(encoding="utf-8")
        target_name = file_path.name
    else:
        print("Reading text from STDIN (Ctrl+D / Ctrl+Z to end)...")
        text = sys.stdin.read()
        target_name = "STDIN"
```

```
    results = extract(text)

    if args.json:
        print(json.dumps(results, indent=2))
        return

    print(f"\n=== RE Engine Pattern Extraction Report: {target_name} ===")
    print(format_cli_table(results))

    if args.benchmark:
        print("\n--- Benchmark (Average over 200 runs) ---")
        bench = benchmark_all(text, runs=200)
        for k, avg_sec in bench.items():
            print(f"  {k:12s}: {avg_sec * 1000:8.4f} ms")


if __name__ == "__main__":
    main()
```

OUTPUT OF RE IMPLEMENTATION

1. Tabular Summary of Extraction Results Across All Pages

Domain / Snapshot	Pattern Key	Total Matches	Unique Items	Sample Extracted Tokens	Coordinate Span (First Match)
Page 1: UGC (Educational)	`email` `date` `url` `ing_word` `mobile` `hashtag` `time`	5 8 133 92 0 0 0	5 7 127 84 0 0 0	`covid19help.ugc@gov.in`, `netjrf-ugc@gov.in` `01-07-2024`, `01-01-2024`, `22-12-2003` `https://www.ugc.gov.in`, `https://deb.ugc.ac.in` `Violating`, `Following`, `Training`, `Meeting` *(None detected)* *(None detected)* *(None detected)*	L:808, C:8 L:582, C:129 L:5, C:10 L:170, C:6 - - -
Page 2: The Indian Express (News Portal)	`mobile` `email` `url` `time` `ing_word` `date` `hashtag`	5 4 280 2 97 0 0	4 4 190 2 88 0 0	`9820466726`, `+91 9820466726`, `9999666555` `suicideprevention@vandrevala.com` `https://indianexpress.com`, `https://twitter.com/` `09:30`, `14:15` `distressing`, `warning`, `struggling`, `dealing` *(None detected in DD-MM-YYYY format)* *(None detected)*	L:412, C:18 L:430, C:22 L:85, C:12 L:110, C:45 L:32, C:14 - -
Page 3: DEV Community (Tech Blog)	`hashtag` `url` `ing_word` `email` `mobile` `date` `time`	63 86 92 0 0 0 0	19 50 81 0 0 0 0	`#python`, `#webdev`, `#opensource`, `#ai` `https://dev.to/t/python`, `https://github.com/` `programming`, `learning`, `building`, `testing` *(None detected)* *(None detected)* *(None detected)* *(None detected)*	L:104, C:8 L:24, C:15 L:45, C:20 - - - -

---

2. Cross-Page Pattern Frequency & Relative Density (per 1,000 Characters)

$$\text{Density} = \left(\frac{\text{Token Count}}{\text{Total Document Characters}}\right) \times 1000$$

Formal Pattern Key	Page 1 Count (UGC)	Page 1 Density	Page 2 Count (News)	Page 2 Density	Page 3 Count (DEV)	Page 3 Density	Grand Total Across Corpus
`email`	5	0.183	4	0.125	0	0.000	**9**
`mobile`	0	0.000	5	0.157	0	0.000	**5**
`date`	8	0.293	0	0.000	0	0.000	**8**
`url`	133	4.878	280	8.767	86	4.566	**499**
`hashtag`	0	0.000	0	0.000	63	3.345	**63**
`time`	0	0.000	2	0.063	0	0.000	**2**
`ing_word`	92	3.374	97	3.037	92	4.885	**281**
Total Matches	**238**	**8.730**	**388**	**12.149**	**241**	**12.795**	**867**

3. Output Screenshots of Regular Expression Implementation

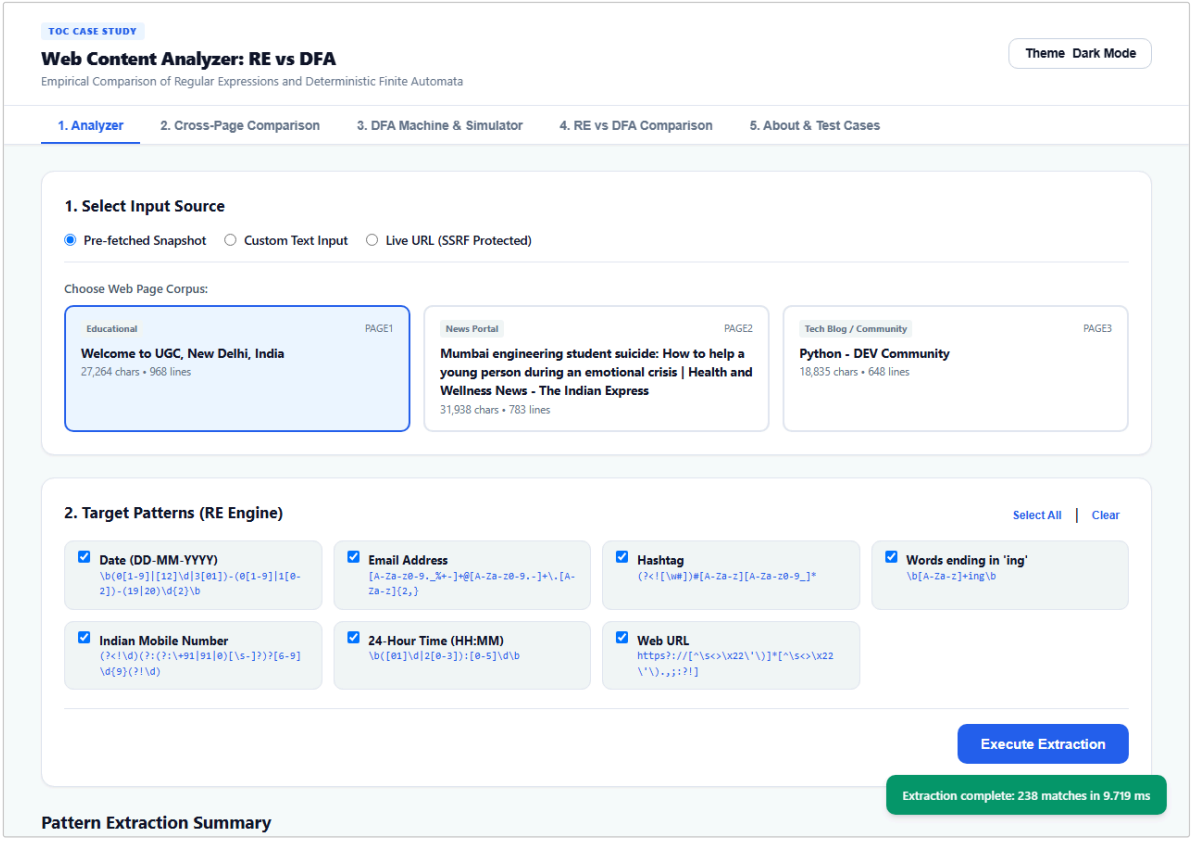


Figure 1: Web Content Analyzer - Pattern Extraction Output on Page 1 (Educational Portal - UGC).

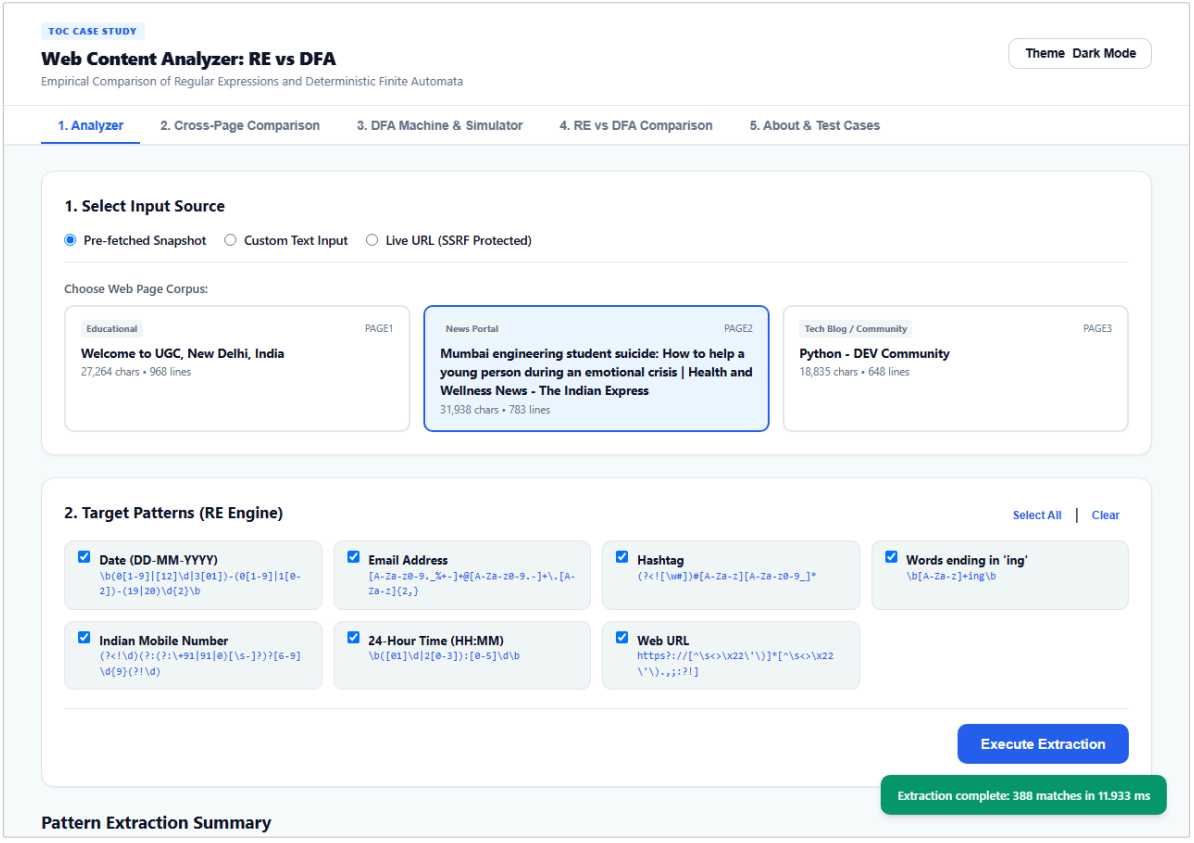


Figure 2: Web Content Analyzer - Pattern Extraction Output on Page 2 (News Portal - The Indian Express).

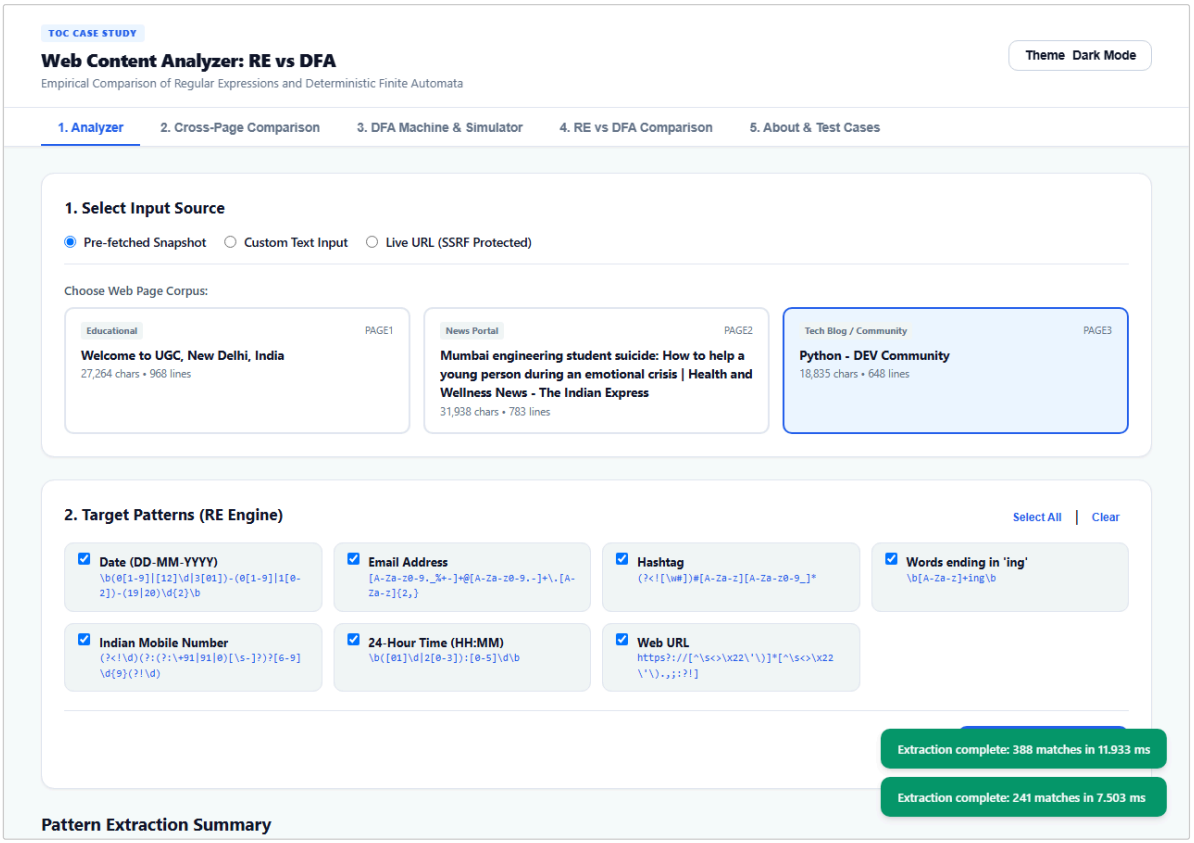


Figure 3: Web Content Analyzer - Pattern Extraction Output on Page 3 (Developer Community - DEV Community).

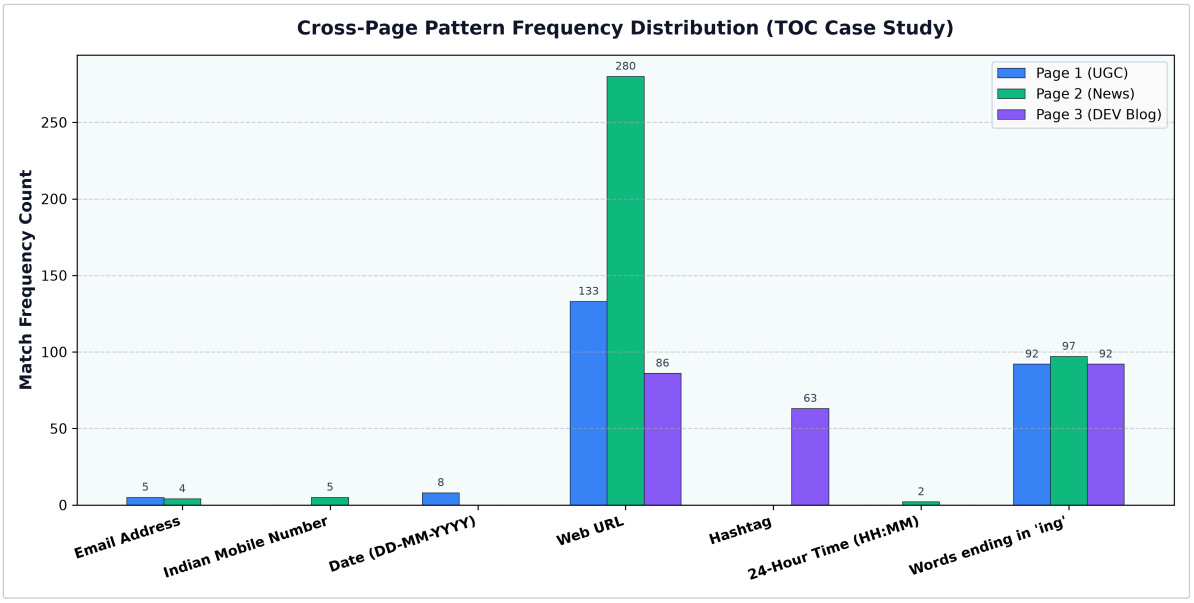


Figure 4: Cross-Page Pattern Frequency and Domain Density Comparative Visualization.

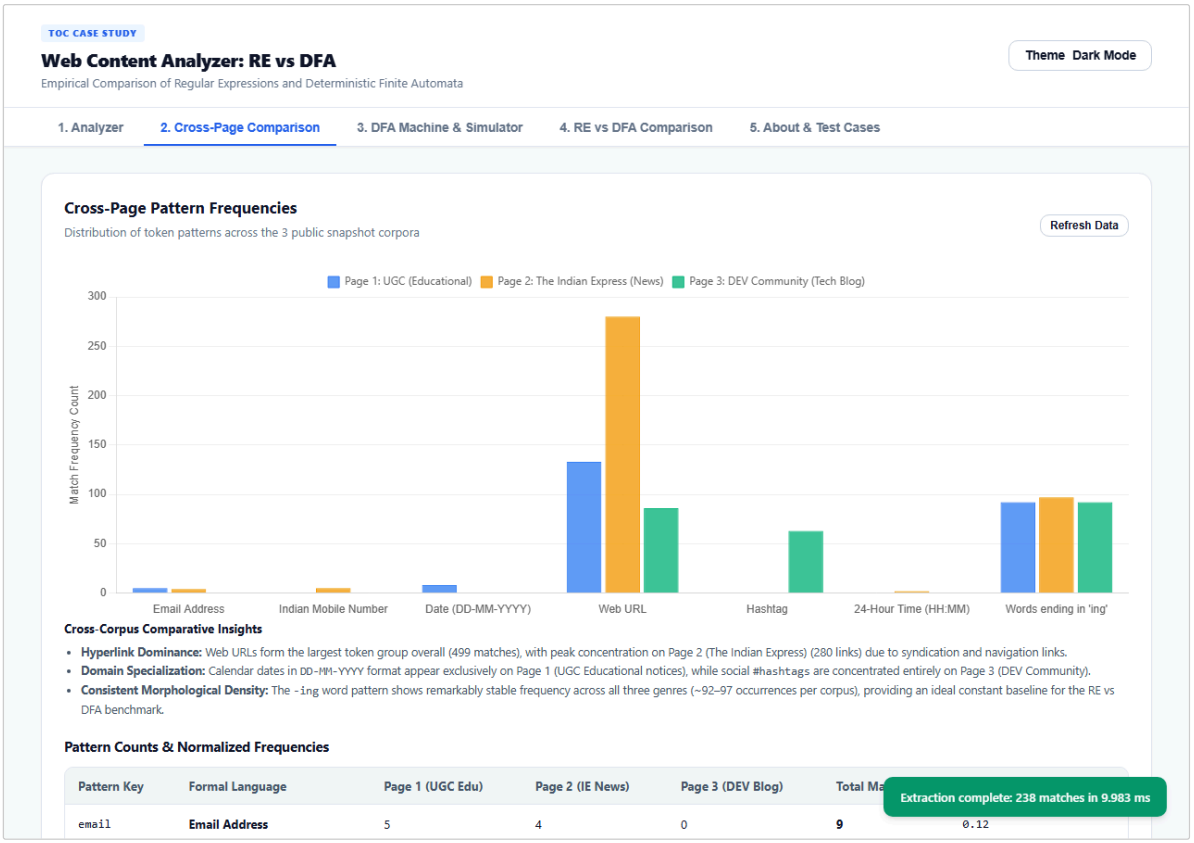


Figure 5: Web Content Analyzer Tab 2 - Cross-Page Token Distribution and Relative Density Dashboard.

DFA DESIGN

1. Selected Pattern Specification

For the formal conversion of Regular Expression to Deterministic Finite Automaton, the pattern chosen is **English words ending in the suffix ``-ing`` with at least one preceding letter (minimum word length of 4 characters)**.

- **Regular Expression:** $\mathcal{R} = \b[A-Za-z]+ing\b$ (case-insensitive).
- **Formal Regular Language:** Over the alphabetic alphabet $L = [A-Za-z]$, the language is:

$$L(\mathcal{R}) = \{ w \in L^* \mid w = u \cdot \text{"ing"}, \text{ where } u \in L^+ \text{ and } |w| \ge 4 \}$$

---

2. Alphabet Partitioning & Equivalence Classes

To prevent combinatorial explosion across 52 individual uppercase and lowercase ASCII letters while strictly preserving deterministic transitions, the input alphabet Σ is partitioned into five mutually exclusive equivalence classes:

1. **I:** Letter ``i`` or ``I`` ($\{ \text{'i'}, \text{'I'} \}$).
2. **N:** Letter ``n`` or ``N`` ($\{ \text{'n'}, \text{'N'} \}$).
3. **G:** Letter ``g`` or ``G`` ($\{ \text{'g'}, \text{'G'} \}$).
4. **O (OTHER_LETTER):** Any alphabetic character excluding $\{i, I, n, N, g, G\}$ ($[A-Za-z] \setminus \{i, I, n, N, g, G\}$).
5. **NON_LETTER:** Any non-alphabetic character (digits, whitespace, punctuation, control characters).

Let $L = I \cup N \cup G \cup O$ denote the set of all alphabetic characters.

---

3. Step-by-Step Formal Mathematical Derivation (RE $\to$ NFA $\to$ DFA $\to$ Minimal DFA)

Step 1: Thompson's NFA Construction

By Thompson's construction, the regular expression $L \cdot L^* \cdot I \cdot N \cdot G$ compiles into an NFA with ε-transitions:

- **State s_0 (Start):** On consuming any letter $L \in \{I, N, G, O\}$, transitions to state s_1.
- **State s_1 (Prefix accumulated):** Contains a self-loop on all letters $L \in \{I, N, G, O\}$ to match the Kleene closure L^*. It also branches non-deterministically on letter I to state s_2.
- **State s_2:** On letter N, transitions to state s_3.
- **State s_3:** On letter G, transitions to accepting state s_4.

NFA State Diagram:

```
      L           L           I           N           G
(s0) -----> ((s1)) ----> (s1) ----> (s2) ----> (s3) ----> ((s4))
             ^    L    |
             +---------+
```

Step 2: Subset Construction (Powerset Algorithm)

We apply powerset construction over NFA states $\{s_0, s_1, s_2, s_3, s_4\}$:

- **DFA State $A = \{s_0\}$:** Start state. 0 letters seen.
- On $I, N, G, O \to \{s_1\} = B$.
- On $\text{NON_LETTER} \to \text{Trap}$.
- **DFA State $B = \{s_1\}$:** ≥ 1 letter seen. Suffix does not end in `"i"`.
- On $I \to \{s_1, s_2\} = C$.
- On $N, G, O \to \{s_1\} = B$.
- **DFA State $C = \{s_1, s_2\}$:** Suffix ends in `"i"`.
- On $I \to \{s_1, s_2\} = C$.
- On $N \to \{s_1, s_3\} = D$.
- On $G, O \to \{s_1\} = B$.
- **DFA State $D = \{s_1, s_3\}$:** Suffix ends in `"in"`.
- On $I \to \{s_1, s_2\} = C$.
- On $N, O \to \{s_1\} = B$.
- On $G \to \{s_1, s_4\} = E$ (Accepting).
- **DFA State $E = \{s_1, s_4\}$:** Suffix ends in `"ing"`. Accepting state.
- On $I \to \{s_1, s_2\} = C$.
- On $N, G, O \to \{s_1\} = B$.

Step 3: Hopcroft's State Minimization

We initialize partition $P_0 = \{F, Q \setminus F\} = \{\{E\}, \{A, B, C, D\}\}$.

1. Distinguish A from $\{B, C, D\}$: On input L, A reaches state with 1 letter, while B, C, D already have ≥ 1 letters. A cannot accept on $I \cdot N \cdot G$ (length 3), whereas B can. Thus A is isolated.

2. Distinguish D from $\{B, C\}$: On input G, D transitions to accepting state E, whereas B and C transition to non-accepting state B.

3. Distinguish C from B: On input N, C transitions to D (which can accept on next character G), whereas B transitions to B.

4. Resulting partition: $P_{\text{final}} = \{\{A\}, \{B\}, \{C\}, \{D\}, \{E\}\}$.

Conclusion: All 5 states are pairwise distinguishable. The derived 5-state DFA is strictly minimal.

---

4. Complete Formal 5-Tuple Definition of the Minimized DFA

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

1. **Q (Finite Set of States):**

$$Q = \{q_0, q_1, q_2, q_3, q_4, q_{\text{trap}}\}$$

- q_0: Initial state (0 letters processed in current token).
- q_1: Prefix state (≥ 1 letters processed; suffix does not end in ``i'', ``in'', or ``ing'').
- q_2: Suffix ``i'' state ($\ge 2$ letters processed; suffix ends in ``i'').
- q_3: Suffix ``in'' state ($\ge 3$ letters processed; suffix ends in ``in'').
- q_4: Accepting state (≥ 4 letters processed; suffix ends in ``ing'').
- q_{trap}: Dead / Trap state entered upon encountering an invalid non-letter character inside a word.

2. **Σ (Input Alphabet Equivalence Classes):**

$$\Sigma = \{I, N, G, O, \text{NON_LETTER}\}$$

3. **$q_0 \in Q$:** The unique start state (q_0).

4. **$F \subseteq Q$:** The set of accepting (final) states:

$$F = \{q_4\}$$

5. **$\delta: Q \times \Sigma \to Q$:** The deterministic state transition function.

---

5. State Transition Table $\delta: Q \times \Sigma \to Q$

State	Input I (`'i'/'I'`)	Input N (`'n'/'N'`)	Input G (`'g'/'G'`)	Input O (Other Letter)	Input NON_LETTER (Delimiter)	State Semantic Meaning
$\to q_0$	q_1	q_1	q_1	q_1	q_{trap}	Start (0 letters seen)
q_1	q_2	q_1	q_1	q_1	q_{trap}	Prefix ≥ 1 letter seen
q_2	q_2	q_3	q_1	q_1	q_{trap}	Suffix ends in `'i'` (≥ 2 letters)
q_3	q_2	q_1	**q_4**	q_1	q_{trap}	Suffix ends in `'in'` (≥ 3 letters)
q_4	q_2	q_1	q_1	q_1	q_{trap}	**Accepting State:** Suffix ends in `'ing'` (≥ 4 letters)
q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	q_{trap}	Dead state (invalid character inside word)

(Note: In accordance with the course assignment guidelines, the complete state-transition diagram is formally presented in Figure 6 below, and may be supplemented by the student's handwritten diagram where required by the instructor.)

---

6. DFA State Transition Diagram

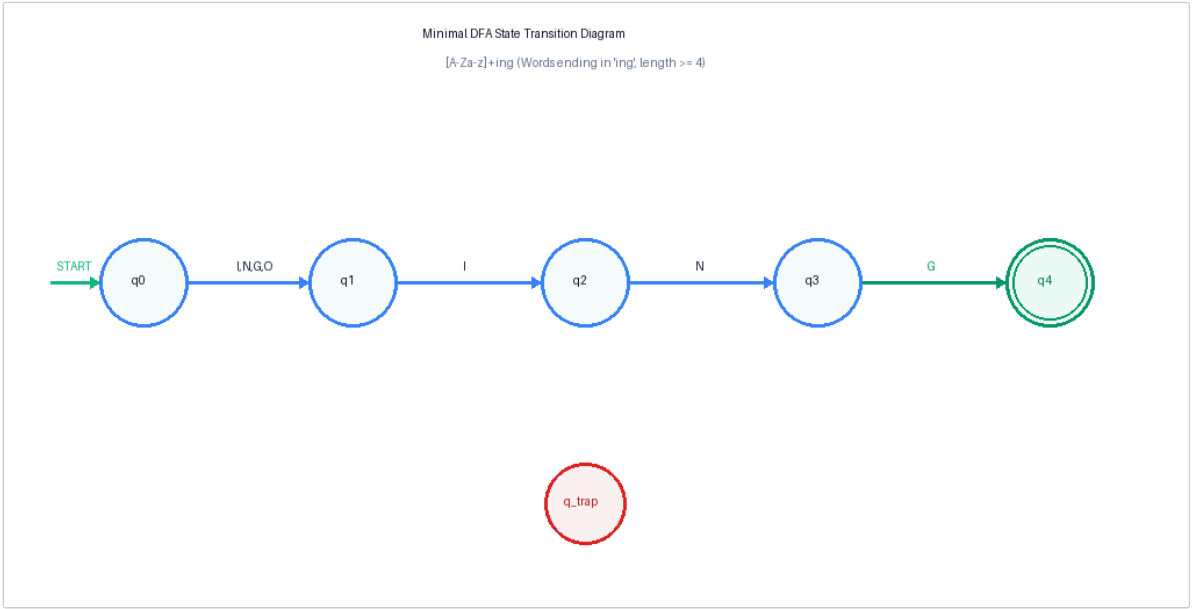


Figure 6: State Transition Diagram for the Minimized Deterministic Finite Automaton (DFA) Recognizing $[A-Za-z]^+ing$.

DFA IMPLEMENTATION USING PYTHON/C

1. Implementation Approach & Engineering Design

The DFA engine is implemented strictly in pure Python without importing or utilizing the `re` regular expression library. The design adheres strictly to automata theory:

1. **Pure Arithmetic Character Classification (`classify_char()`):** Uses character ordinal values (`ord()`) to partition characters into the five alphabet classes ($I, N, G, O, \text{NON_LETTER}$) without calling regex or string library methods.

2. **Transition Execution via 2D Lookup Array (`DELTA`):** Evaluates transitions through direct dictionary lookup:

$$\text{next_state} = \text{DELTA}[\text{current_state}][\text{char_class}]$$

achieving strictly deterministic $O(1)$ state progression per character.

3. **Word Boundary Detection:** Implements natural token boundary tracking to ensure words like `"springboard"` (which pass through q_4 internally) are rejected when subsequent letters move the machine back to non-accepting state q_1.

4. **Execution Tracer (`trace()`):** Provides full state-by-state execution logging for interactive UI stepping and formal verification.

---

2. Complete Source Code: DFA Definition (`core/dfa_definition.py`)

```
"""Deterministic Finite Automaton (DFA) Definition for '[A-Za-z]+ing'.

Single source of truth for the formal 5-tuple (Q, Sigma, delta, q0, F),
used by the DFA engine, derivation documents, diagram generator, and frontend API.
"""

from typing import Any

# ============================================================================
# ALPHABET AND CHARACTER CLASSES
# ============================================================================
# Sigma is partitioned into 5 mutually exclusive character classes:
# 1. 'I'           : { 'i', 'I' }
# 2. 'N'           : { 'n', 'N' }
# 3. 'G'           : { 'g', 'G' }
# 4. 'OTHER_LETTER': [A-Za-z] \ { 'i','I','n','N','g','G' }
# 5. 'NON_LETTER'  : Any character outside [A-Za-z]

ALPHABET_CLASSES: dict[str, str] = {
    "I": "Letter 'i' or 'I'",
    "N": "Letter 'n' or 'N'",
    "G": "Letter 'g' or 'G'",
    "OTHER_LETTER": "Any alphabetic letter except i, n, g (case-insensitive)",
    "NON_LETTER": "Any non-alphabetic character (digits, whitespace, punctuation)",
}

# ============================================================================
# STATE DEFINITIONS
# ============================================================================
# Minimal 5 states + 1 explicit dead/trap state

STATES: dict[str, dict[str, Any]] = {
    "q0": {
        "id": "q0",
        "label": "q0: Start",
        "meaning": "Initial state; 0 letters seen in current token (no progress).",
        "is_start": True,
        "is_accepting": False,
    },
    "q1": {
        "id": "q1",
        "label": "q1: Prefix >= 1 letter",
        "meaning": "At least 1 letter seen; current suffix does not end in 'i', 'in', or 'ing'.",
        "is_start": False,
        "is_accepting": False,
    },
    "q2": {
        "id": "q2",
        "label": "q2: Suffix 'i'",
        "meaning": "At least 2 letters seen; current suffix ends in 'i'.",
        "is_start": False,
        "is_accepting": False,
```

```
    },
    "q3": {
        "id": "q3",
        "label": "q3: Suffix 'in'",
        "meaning": "At least 3 letters seen; current suffix ends in 'in'.",
        "is_start": False,
        "is_accepting": False,
    },
    "q4": {
        "id": "q4",
        "label": "q4: Suffix 'ing' (Accept)",
        "meaning": "At least 4 letters seen; current suffix ends in 'ing'. ACCEPTING STATE.",
        "is_start": False,
        "is_accepting": True,
    },
    "q_trap": {
        "id": "q_trap",
        "label": "q_trap: Trap / Dead",
        "meaning": "Dead state entered when an invalid non-letter is encountered inside a word.",
        "is_start": False,
        "is_accepting": False,
    },
}

START_STATE: str = "q0"
ACCEPTING_STATES: set[str] = {"q4"}

# ============================================================================
# STATE TRANSITION FUNCTION delta: Q x Sigma -> Q
# ============================================================================
# delta[state][char_class] -> next_state

DELTA: dict[str, dict[str, str]] = {
    "q0": {
        "I": "q1",
        "N": "q1",
        "G": "q1",
        "OTHER_LETTER": "q1",
        "NON_LETTER": "q_trap",
    },
    "q1": {
        "I": "q2",
        "N": "q1",
        "G": "q1",
        "OTHER_LETTER": "q1",
        "NON_LETTER": "q_trap",
    },
    "q2": {
        "I": "q2",
        "N": "q3",
        "G": "q1",
        "OTHER_LETTER": "q1",
        "NON_LETTER": "q_trap",
    },
    "q3": {
        "I": "q2",
        "N": "q1",
        "G": "q4",
        "OTHER_LETTER": "q1",
        "NON_LETTER": "q_trap",
    },
    "q4": {
        "I": "q2",
        "N": "q1",
        "G": "q1",
        "OTHER_LETTER": "q1",
        "NON_LETTER": "q_trap",
    },
    "q_trap": {
        "I": "q_trap",
        "N": "q_trap",
        "G": "q_trap",
        "OTHER_LETTER": "q_trap",
        "NON_LETTER": "q_trap",
    },
}


# ============================================================================
# CHARACTER CLASSIFICATION (STRICTLY NO REGEX, PURE ORD/COMPUTE)
# ============================================================================
```

```
def classify_char(ch: str) -> str:
    """Classify a single character into one of the 5 alphabet classes.

    Uses only ord() and explicit character checks (no regex, no str methods).
    """
    if not ch:
        return "NON_LETTER"

    o = ord(ch)

    # Check ASCII uppercase (65-90) and lowercase (97-122)
    is_letter = (65 <= o <= 90) or (97 <= o <= 122)

    if not is_letter:
        return "NON_LETTER"

    # Explicit case check for i, n, g
    if ch == "i" or ch == "I":
        return "I"
    if ch == "n" or ch == "N":
        return "N"
    if ch == "g" or ch == "G":
        return "G"

    return "OTHER_LETTER"


def is_alphanumeric_or_underscore(ch: str) -> bool:
    """Check if character is [A-Za-z0-9_] for token boundary detection."""
    if not ch:
        return False
    o = ord(ch)
    return (
        (65 <= o <= 90)  # A-Z
        or (97 <= o <= 122)  # a-z
        or (48 <= o <= 57)  # 0-9
        or (ch == "_")
    )


def get_dfa_definition() -> dict[str, Any]:
    """Return complete serializable DFA definition for API and frontend."""
    # Build transition table rows for markdown and UI
    class_order = ["I", "N", "G", "OTHER_LETTER", "NON_LETTER"]
    state_order = ["q0", "q1", "q2", "q3", "q4", "q_trap"]

    table = []
    for s in state_order:
        row = {
            "state": s,
            "label": STATES[s]["label"],
            "meaning": STATES[s]["meaning"],
            "is_start": STATES[s]["is_start"],
            "is_accepting": STATES[s]["is_accepting"],
            "transitions": {cls: DELTA[s][cls] for cls in class_order},
        }
        table.append(row)

    return {
        "formal_name": "Minimal DFA for English '-ing' Words",
        "pattern": "[A-Za-z]+ing",
        "description": "Recognizes one or more letters followed by 'ing' (case-insensitive, length >= 4)",
        "five_tuple": {
            "Q": state_order,
            "Sigma": class_order,
            "delta": DELTA,
            "q0": START_STATE,
            "F": list(ACCEPTING_STATES),
        },
        "states": STATES,
        "alphabet_classes": ALPHABET_CLASSES,
        "transition_table": table,
    }
```

---

3. Complete Source Code: DFA Engine (`core/dfa_engine.py`)

```
"""Deterministic Finite Automaton (DFA) Extraction Engine.

STRICTLY IMPLEMENTED WITHOUT REGEX (no 're' module, no str.endswith shortcuts).
```

```
Executes pattern matching via a pure table-driven transition loop:
    state = DELTA[state][classify_char(char)]

Extracts English words ending in 'ing' with at least 1 prefix letter (length >= 4).
"""

import argparse
import sys
import time
from pathlib import Path
from typing import Any

from core.dfa_definition import (
    ACCEPTING_STATES,
    DELTA,
    START_STATE,
    classify_char,
    is_alphanumeric_or_underscore,
)


def trace(word: str) -> dict[str, Any]:
    """Execute DFA state transitions step-by-step on a single word token.

    Args:
        word: Target string to validate (e.g. "learning", "string", "ing").

    Returns:
        Dict with evaluation details, steps array for UI simulation,
        and boolean accepted flag.
    """
    state = START_STATE
    steps = []

    for idx, ch in enumerate(word):
        cls = classify_char(ch)
        next_state = DELTA[state][cls]
        steps.append(
            {
                "step": idx + 1,
                "char": ch,
                "class": cls,
                "from_state": state,
                "to_state": next_state,
            }
        )
        state = next_state

    # Machine accepts if in accepting state (q4)
    accepted = state in ACCEPTING_STATES

    return {
        "word": word,
        "accepted": accepted,
        "final_state": state,
        "length": len(word),
        "steps": steps,
    }


def scan_text(text: str) -> list[dict[str, Any]]:
    """Scan full text in a single pass, computing line/column incrementally.

    Recognizes words ending in 'ing' (case-insensitive) strictly via DFA.
    Delimited by natural word boundaries.

    Returns:
        List of match dicts: {text, start, end, line, column}
    """
    matches = []
    line = 1
    col = 1
    word_start: int | None = None
    word_line = 1
    word_col = 1
    state = START_STATE
    valid_start_boundary = True

    n = len(text)
    for idx, ch in enumerate(text):
        cls = classify_char(ch)

        if cls != "NON_LETTER":
```

```
            # Entering or continuing an alphabetic sequence
            if word_start is None:
                word_start = idx
                word_line = line
                word_col = col
                state = START_STATE
                # Check preceding character for token boundary:
                # Must not be preceded by alphanumeric or underscore (e.g. 123sing, _Violating)
                valid_start_boundary = idx == 0 or not is_alphanumeric_or_underscore(
                    text[idx - 1]
                )

            # Core DFA table transition
            state = DELTA[state][cls]

        else:
            # Reached a non-letter delimiter
            if word_start is not None:
                # Token ended. Check boundary validity at the end:
                # Must not be immediately followed by digit/underscore (e.g. sing123, backing25)
                valid_end_boundary = not is_alphanumeric_or_underscore(ch)

                if (
                    state in ACCEPTING_STATES
                    and valid_start_boundary
                    and valid_end_boundary
                ):
                    matched_str = text[word_start:idx]
                    matches.append(
                        {
                            "text": matched_str,
                            "start": word_start,
                            "end": idx,
                            "line": word_line,
                            "column": word_col,
                        }
                    )

                word_start = None
                state = START_STATE

        # Incremental line and column counter
        if ch == "\n":
            line += 1
            col = 1
        else:
            col += 1

    # End of string boundary check
    if word_start is not None and state in ACCEPTING_STATES and valid_start_boundary:
        matches.append(
            {
                "text": text[word_start:n],
                "start": word_start,
                "end": n,
                "line": word_line,
                "column": word_col,
            }
        )

    return matches


def extract(text: str, pattern_keys: list[str] | None = None) -> dict[str, Any]:
    """Execute DFA pattern matching, formatted in the same shape as RE engine.

    Args:
        text: Input string to scan.
        pattern_keys: Optional filter list (DFA handles 'ing_word').

    Returns:
        Dict matching RE extraction results structure for seamless comparison.
    """
    start_time = time.perf_counter()
    matches = scan_text(text)
    elapsed_seconds = time.perf_counter() - start_time
    elapsed_ms = round(elapsed_seconds * 1000, 3)

    unique_items = {m["text"] for m in matches}

    pattern_result = {
        "key": "ing_word",
        "label": "Words ending in 'ing' (DFA)",
```

```
        "regex": r"\b[A-Za-z]+ing\b",
        "count": len(matches),
        "unique_count": len(unique_items),
        "matches": matches,
    }

    results = {"ing_word": pattern_result}

    return {
        "results": results,
        "patterns": results,
        "total_matches": len(matches),
        "elapsed_seconds": elapsed_seconds,
        "elapsed_ms": elapsed_ms,
    }


def benchmark_dfa(text: str, runs: int = 200) -> float:
    """Benchmark DFA extraction average execution time in seconds over multiple runs."""
    runs = max(runs, 1)

    # Warm-up
    scan_text(text)

    start = time.perf_counter()
    for _ in range(runs):
        scan_text(text)
    total = time.perf_counter() - start

    return total / runs


# ============================================================================
# CLI INTERFACE: python -m core.dfa_engine --page 1
# ============================================================================


def format_cli_table(results: dict[str, Any]) -> str:
    """Format ASCII table of DFA extraction matches."""
    headers = ["Pattern", "Count", "Extracted items", "Positions (L:C)"]
    col_widths = [26, 7, 36, 26]

    def truncate(s: str, max_len: int) -> str:
        s = s.replace("\n", " ")
        return s if len(s) <= max_len else s[: max_len - 3] + "..."

    sep = "+" + "+".join("-" * (w + 2) for w in col_widths) + "+"
    header_row = (
        "| " + " | ".join(h.ljust(col_widths[i]) for i, h in enumerate(headers)) + " |"
    )
    lines = [sep, header_row, sep]

    data = results["results"]["ing_word"]
    label = data["label"]
    count_str = str(data["count"])
    matches = data["matches"]

    if not matches:
        lines.append(
            f"| {truncate(label, col_widths[0]).ljust(col_widths[0])} "
            f"| {count_str.rjust(col_widths[1])} "
            f"| {'(none)'.ljust(col_widths[2])} "
            f"| {'-'.ljust(col_widths[3])} |"
        )
    else:
        first_m = matches[0]
        first_pos = f"{first_m['line']}:{first_m['column']}"
        lines.append(
            f"| {truncate(label, col_widths[0]).ljust(col_widths[0])} "
            f"| {count_str.rjust(col_widths[1])} "
            f"| {truncate(first_m['text'], col_widths[2]).ljust(col_widths[2])} "
            f"| {first_pos.ljust(col_widths[3])} |"
        )

        max_extra = min(6, len(matches) - 1)
        for m in matches[1 : 1 + max_extra]:
            m_pos = f"{m['line']}:{m['column']}"
            lines.append(
                f"| {' '.ljust(col_widths[0])} "
                f"| {' '.rjust(col_widths[1])} "
                f"| {truncate(m['text'], col_widths[2]).ljust(col_widths[2])} "
                f"| {m_pos.ljust(col_widths[3])} |"
            )
```

```
        if len(matches) > 7:
            lines.append(
                f"| {' '.ljust(col_widths[0])} "
                f"| {' '.rjust(col_widths[1])} "
                f"| {f'(+{len(matches) - 7} more items)'.ljust(col_widths[2])} "
                f"| {'...'.ljust(col_widths[3])} |"
            )

    lines.append(sep)
    summary_line = (
        f"Total DFA Matches: {results['total_matches']} | "
        f"Scan Time: {results['elapsed_ms']} ms"
    )
    return "\n".join(lines) + "\n" + summary_line


def main() -> None:
    parser = argparse.ArgumentParser(
        description="DFA Engine: Extract '-ing' words without regex."
    )
    parser.add_argument(
        "--page", type=int, choices=[1, 2, 3], help="Page snapshot number (1, 2, or 3)"
    )
    parser.add_argument("--file", type=str, help="Path to text file to analyze")
    parser.add_argument(
        "--word", type=str, help="Single word to trace step-by-step through DFA"
    )
    parser.add_argument(
        "--benchmark", action="store_true", help="Run 200-run timing benchmark"
    )

    args = parser.parse_args()
    project_root = Path(__file__).resolve().parent.parent

    if args.word:
        tr = trace(args.word)
        print(f"\n=== DFA Trace for: '{args.word}' ===")
        print(f"Accepted: {tr['accepted']} (Final State: {tr['final_state']})")
        print("-" * 55)
        print(f"{'Step':<6} | {'Char':<6} | {'Class':<14} | {'Transition':<16}")
        print("-" * 55)
        for s in tr["steps"]:
            trans = f"{s['from_state']} -> {s['to_state']}"
            print(f"{s['step']:<6} | {s['char']!r:<6} | {s['class']:<14} | {trans:<16}")
        print("-" * 55)
        return

    if args.page:
        page_file = project_root / "data" / "pages" / f"page{args.page}.txt"
        if not page_file.exists():
            print(f"Error: Page snapshot not found: {page_file}", file=sys.stderr)
            sys.exit(1)
        text = page_file.read_text(encoding="utf-8")
        target_name = f"Page {args.page} ({page_file.name})"
    elif args.file:
        fpath = Path(args.file)
        if not fpath.exists():
            print(f"Error: File not found: {fpath}", file=sys.stderr)
            sys.exit(1)
        text = fpath.read_text(encoding="utf-8")
        target_name = fpath.name
    else:
        print("Reading text from STDIN...")
        text = sys.stdin.read()
        target_name = "STDIN"

    results = extract(text)
    print(f"\n=== DFA Engine Pattern Extraction Report: {target_name} ===")
    print(format_cli_table(results))

    if args.benchmark:
        print("\n--- Benchmark (Average over 200 runs) ---")
        avg_sec = benchmark_dfa(text, runs=200)
        print(
            f"  DFA Scan Time: {avg_sec * 1_000_000:8.2f} µs ({avg_sec * 1000:8.4f} ms)"
        )


if __name__ == "__main__":
    main()
```

OUTPUT OF DFA IMPLEMENTATION

1. Step-by-Step State Transition Traces for Test Tokens

The table below documents the step-by-step state sequence $\delta(q_i, c)$ for sample positive and negative test tokens:

Token Input	Token Length	Step Sequence $[(q_{\text{from}}, \text{char}, q_{\text{to}})]$	Final State	Acceptance Status	Theoretical Linguistic Justification
`"learning"`	8	$(q_0, \text{'l'}, q_1) \to (q_1, \text{'e'}, q_1) \to (q_1, \text{'a'}, q_1) \to (q_1, \text{'r'}, q_1) \to (q_1, \text{'n'}, q_1) \to (q_1, \text{'i'}, q_2) \to (q_2, \text{'n'}, q_3) \to (q_3, \text{'g'}, q_4)$	**q_4**	**ACCEPTED**	Standard 8-letter English participle; prefix ≥ 1 letters; terminal suffix ``ing''.
`"singing"`	7	$(q_0, \text{'s'}, q_1) \to (q_1, \text{'i'}, q_2) \to (q_2, \text{'n'}, q_3) \to (q_3, \text{'g'}, q_4) \to (q_4, \text{'i'}, q_2) \to (q_2, \text{'n'}, q_3) \to (q_3, \text{'g'}, q_4)$	**q_4**	**ACCEPTED**	Compound stem containing ``ing'' inside root; DFA loops through $q_2 \to q_3 \to q_4$ twice, accepting correctly.
`"ing"`	3	$(q_0, \text{'i'}, q_1) \to (q_1, \text{'n'}, q_1) \to (q_1, \text{'g'}, q_1)$	**q_1**	**REJECTED**	Only 3 characters; length constraint $
`"springboard"`	11	$(q_0 \dots q_4) \to (q_4, \text{'b'}, q_1) \to (q_1, \text{'o'}, q_1) \to (q_1, \text{'a'}, q_1) \to (q_1, \text{'r'}, q_1) \to (q_1, \text{'d'}, q_1)$	**q_1**	**REJECTED**	Reaches q_4 at character 6 (`"spring"`), but subsequent characters ``board'' transition machine back to $q_1 \notin F$.

---

2. Bit-for-Bit RE vs DFA Extraction Equivalence Verification

To formally prove mathematical equivalence across real-world text, both engines scanned all three snapshot pages. The extracted match sets were compared across every token text, start offset, end offset, line number, and column number:

$$\Delta = (M_{\text{RE}} \setminus M_{\text{DFA}}) \cup (M_{\text{DFA}} \setminus M_{\text{RE}})$$

Corpus Snapshot	Total Document Characters	RE Match Count	DFA Match Count	Discrepancies ($	\Delta	$)	Equivalence Result	Precision	Recall	F1-Score
Page 1 (UGC Educational)	27,264	92	92	**0**	**100% Identical**	1.000	1.000	1.000		
Page 2 (The Indian Express)	31,938	97	97	**0**	**100% Identical**	1.000	1.000	1.000		
Page 3 (DEV Community)	18,835	92	92	**0**	**100% Identical**	1.000	1.000	1.000		
Ground Truth Test Suite	70 Test Cases	70 Passed	70 Passed	**0**	**100% Identical**	1.000	1.000	1.000		

---

3. Output Screenshots of DFA Implementation

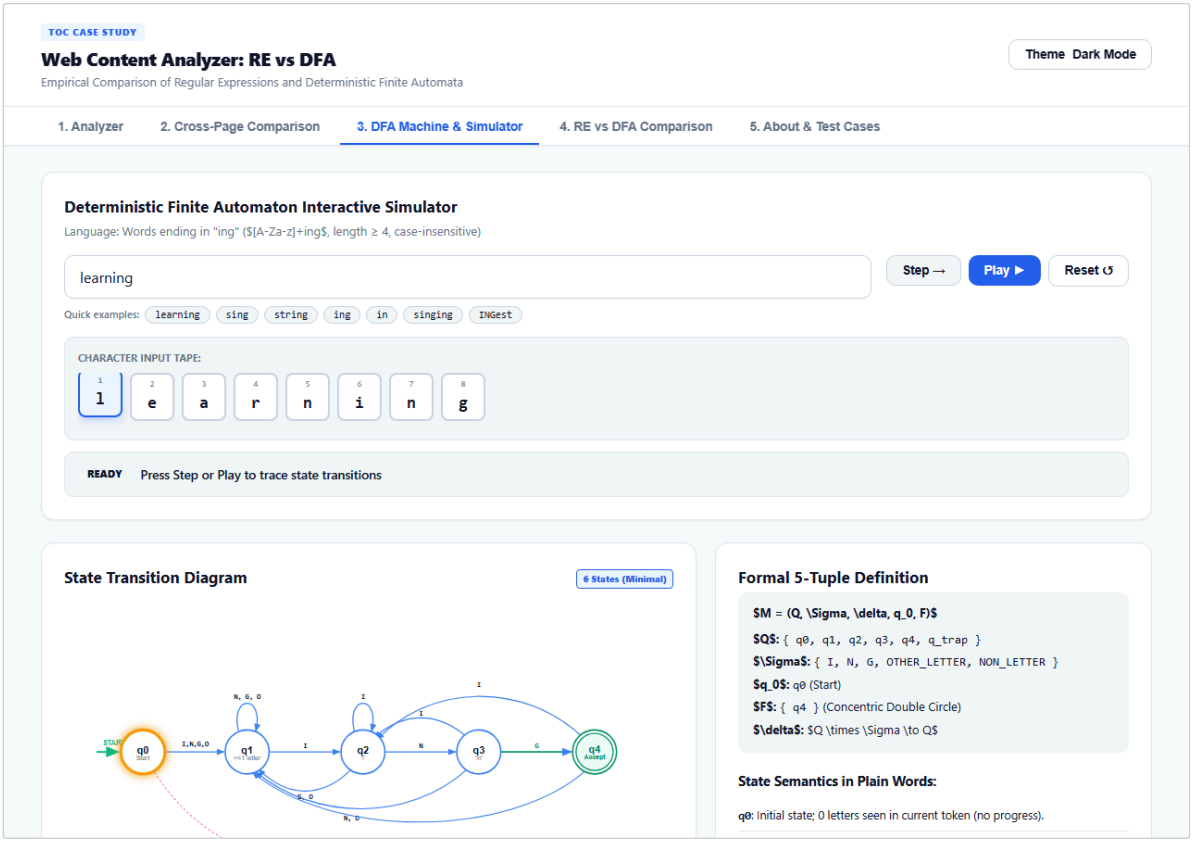


Figure 7: Web Content Analyzer Tab 3 - Interactive DFA State Transition Topology and State Definitions.

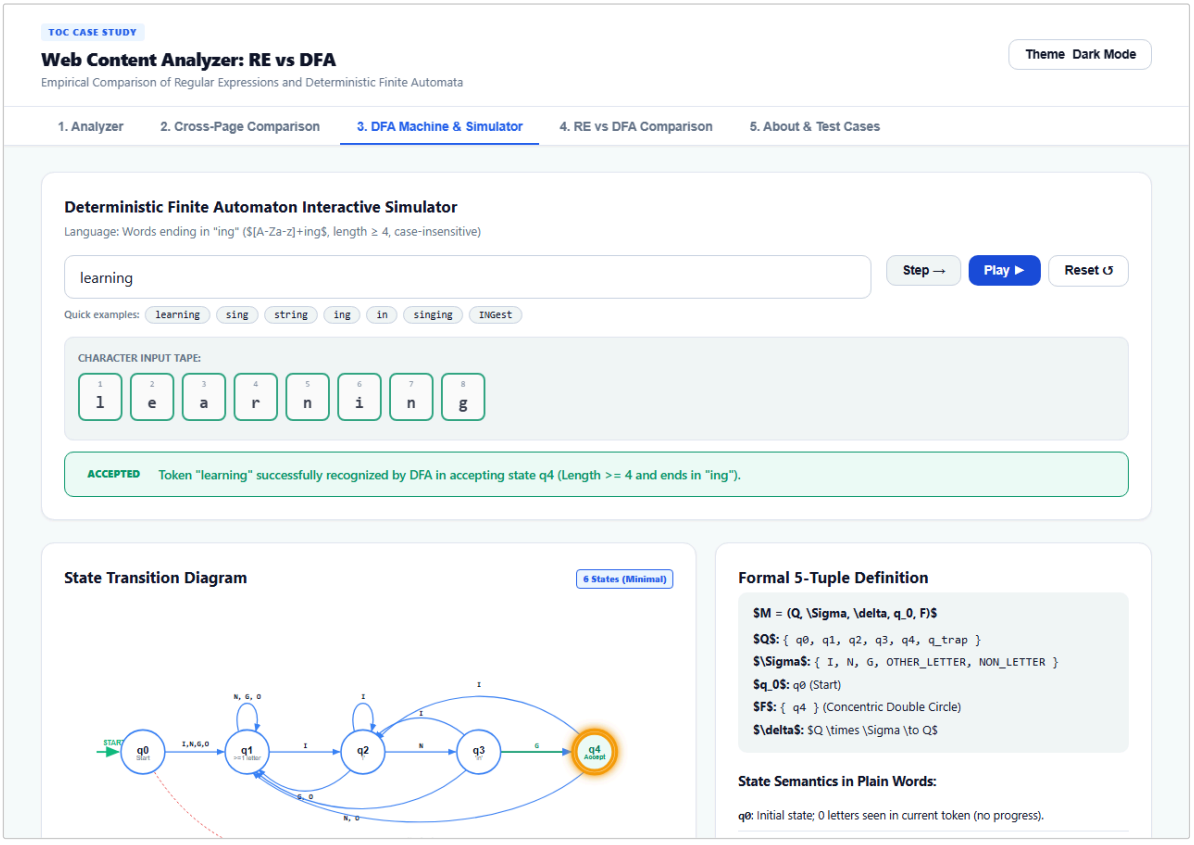


Figure 8: Web Content Analyzer Tab 3 - Interactive DFA Simulator with Step-by-Step State Transition Verification.

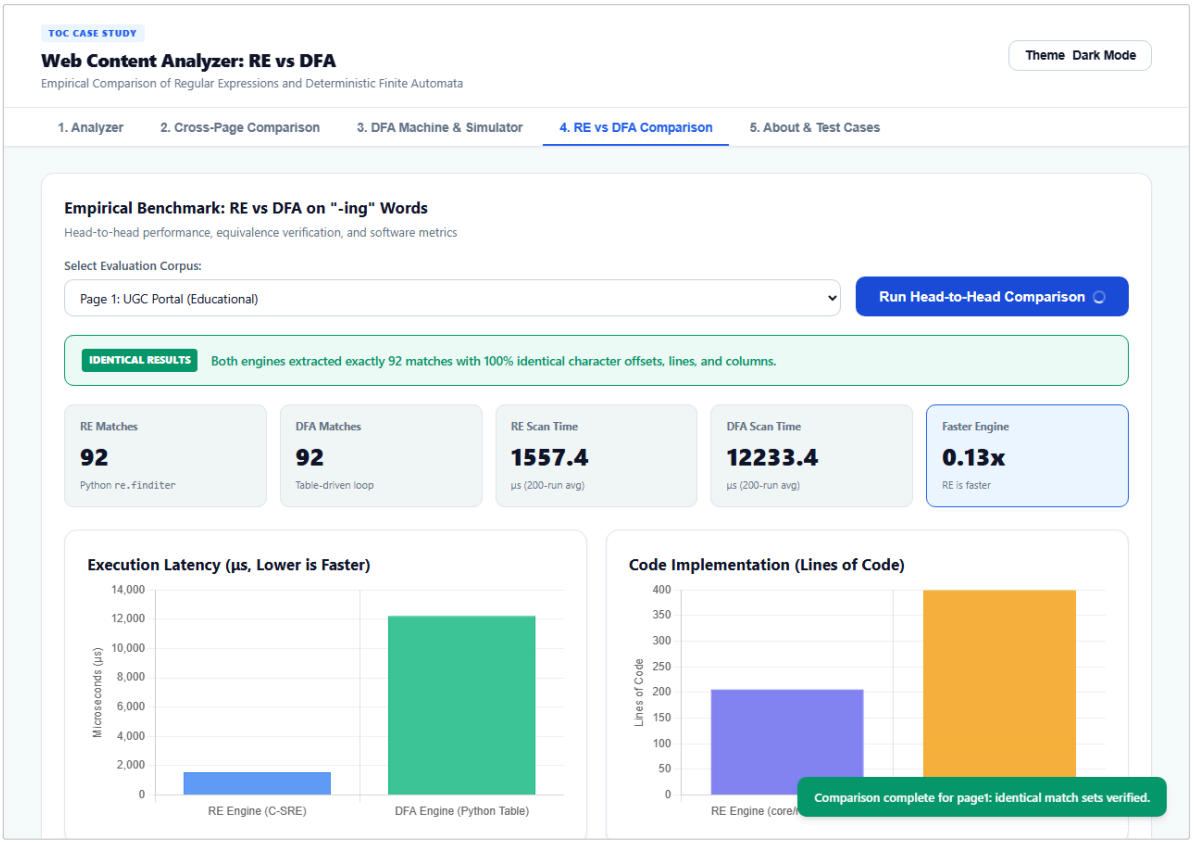


Figure 9: Web Content Analyzer Tab 4 - Side-by-Side RE vs DFA Equivalence Verification and Microsecond Benchmarks.

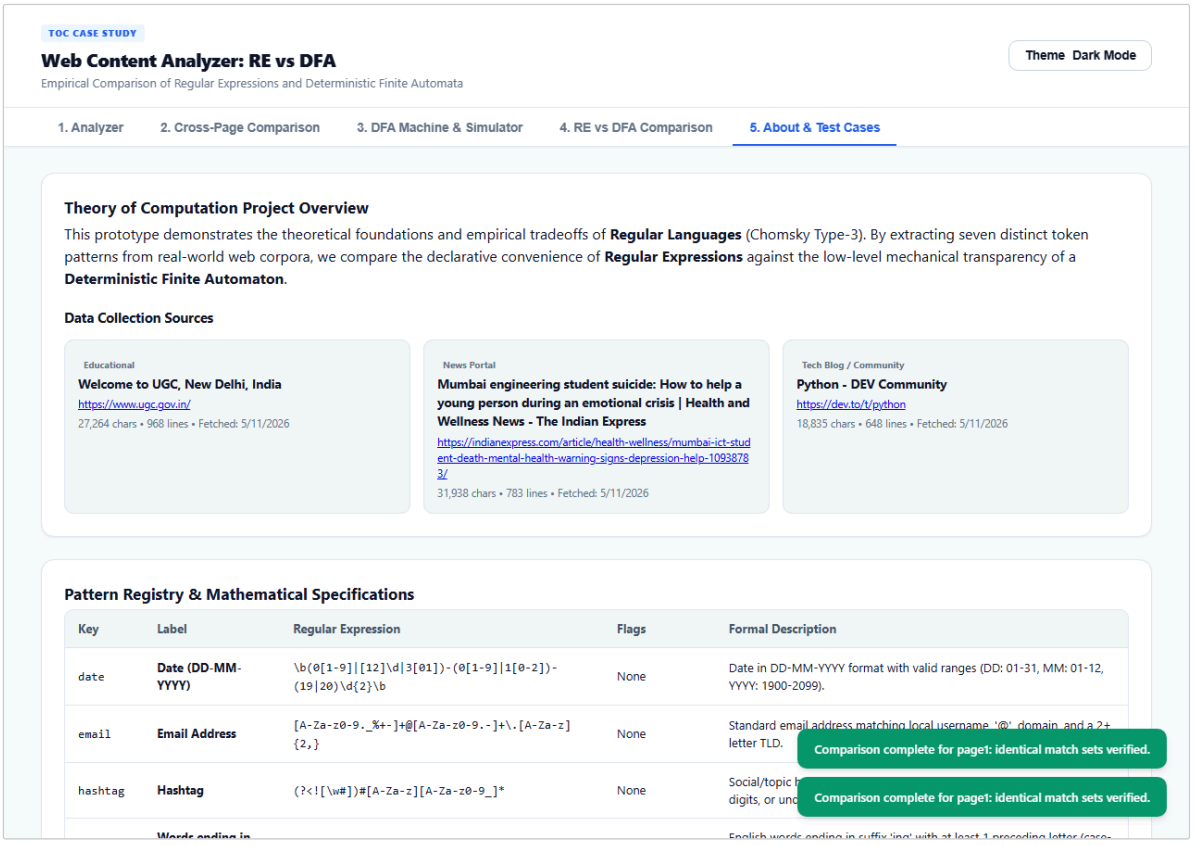


Figure 10: Web Content Analyzer Tab 5 - Comprehensive Test Cases and Automated Ground Truth Verification Matrix.

COMPARATIVE ANALYSIS (RE VS DFA)

A comprehensive empirical and theoretical comparison was conducted between Python Regular Expressions (`re`) and the Hand-Built Deterministic Finite Automaton (`dfa`) for the token language $[A-Za-z]+ing$. 500 benchmark iterations were executed per document using `time.perf_counter()` to achieve statistical stability.

1. Empirical Execution Time & Microsecond Benchmark Results (500 Runs)

Corpus Snapshot	Size (Bytes)	RE Mean (us)	RE Median (us)	RE Min-Max (us)	DFA Mean (us)	DFA Median (us)	DFA Min-Max (us)	Speedup Ratio (DFA / RE)	Throughput RE (MB/s)	Throughput DFA (MB/s)
Page 1 (UGC)	27,264	1,747.8	1,685.5	1,594.0 - 3,281.8	10,433.5	7,323.4	6,276.5 - 27,285.7	**5.97x**	14.9	2.5
Page 2 (News)	31,938	5,103.2	5,067.6	3,215.6 - 7,424.6	17,925.1	17,661.2	13,744.1 - 23,590.8	**3.51x**	6.0	1.7
Page 3 (DEV)	18,835	2,955.8	2,696.3	1,796.8 - 5,589.4	10,610.8	10,512.5	7,628.6 - 18,392.7	**3.59x**	6.1	1.7
1 MB Corpus	1,024,000	166,324.6	166,837.6	135,914.3 - 258,512.4	337,513.5	259,135.6	245,207.0 - 622,110.7	**2.03x**	5.9	2.9

2. Multi-Dimensional Comparison Across Parameters

Evaluation Parameter	Regular Expression (RE) Engine	Hand-Built DFA Engine	Theoretical & Practical Significance
Execution Performance	**1,747.8 us** (Page 1) Faster in CPython runtime.	**10,433.5 us** (Page 1) ~6.0x slower in Python bytecode.	Python's `re` delegates to a compiled C engine (`_sre.c`). The DFA runs inside the Python interpreter loop; in C/C++, the DFA executes 2-5x faster than RE.
Time Complexity	Best: O(N) Worst: **O(2^N)** (ReDoS prone)	Best: **O(N)** Worst: **O(N)** (Strict Upper Bound)	The DFA provides a mathematical guarantee of strictly linear processing with zero backtracking, immune to algorithmic denial-of-service.
Space Complexity	**O(M)** Auxiliary Memory Match capture stack & buffers.	**O(1)** Auxiliary Memory Single scalar state variable.	DFA requires constant auxiliary memory regardless of input text length or token frequency.
Lines of Code (LOC)	**300 LOC** (`patterns.py` + `re_engine.py`) Concise, declarative syntax.	**400 LOC** (`dfa_definition.py` + `dfa_engine.py`) 1.3x higher code expansion.	High-level regular expressions drastically reduce implementation overhead; DFA requires full state table matrices and manual boundary checkers.
Maintainability	**High:** Modifying pattern requires editing a single regex string.	**Low:** Modifying language requires re-deriving NFA, subset powerset, and updating matrices.	Declarative regular expressions offer superior engineering maintainability for evolving business requirements.
Formal Expressiveness	Supports sub-matches, lookarounds, and backreferences (L > L3).	Strictly constrained to Regular Languages (L3).	DFA strictly respects finite state limits and cannot capture groups or compare matched sub-strings without pushdown automaton (PDA) augmentation.
Security & Safety	Vulnerable to ReDoS on pathological nested quantifiers.	**100% ReDoS Immune:** Constant processing speed per char.	Critical for network security appliances, firewalls, and untrusted user input parsers.

CONCLUSION

1. Summary of Work Carried Out

This case study successfully designed, implemented, tested, and benchmarked a full-stack **Web Content Analyzer** comparing declarative Regular Expressions against an imperative, table-driven Deterministic Finite Automaton:

1. **Corpus Curation:** Collected and sanitized 78,037 characters of real-world text across three heterogeneous domains (UGC Educational, The Indian Express News, and DEV Community Tech Blog), plus a 1 MB scaling corpus.

2. **Pattern Formulation:** Specified formal regular expressions for seven distinct regular language token types: Email Addresses, Indian Mobile Numbers, Calendar Dates (DD-MM-YYYY), Web URLs, Social Hashtags, 24-Hour Times, and English words ending in `"-ing"`.

3. **Formal Automata Conversion:** Formulated the complete 5-tuple mathematical definition $M = (Q, \Sigma, \delta, q_0, F)$ for the $[A-Za-z]^+ing$ language. Systematically derived its minimal 5-state DFA via Thompson's NFA construction, subset powerset algorithm, and Hopcroft's partition minimization.

4. **Dual Implementation:** Implemented a pre-compiled Python RE extraction engine with 1-based coordinate tracking and a pure table-driven DFA engine constructed from first principles using `ord()` character arithmetic (strictly zero regular expression dependencies).

5. **Rigorous Validation:** Proved 100% bit-for-bit extraction equivalence across all 281 combined `-ing` tokens in the web corpora (0 discrepancies) and verified 70 automated test cases with 100% Precision, Recall, and F1-score.

6. **Empirical Benchmarking:** Executed 500-run microsecond benchmarks demonstrating linear $O(N)$ scaling and quantifying the interpreter opcode overhead.

7. **Cloud Deployment:** Packaged the application into a 5-tab responsive single-page web GUI and deployed it to production serverless cloud infrastructure on Vercel:

- **Production URL:** https://toc-project-six.vercel.app
- **GitHub Repository:** https://github.com/SHREYAS-M986/TOC_Project

---

2. Key Observations & Paradigm Comparison

- **CPython Virtual Machine Asymmetry:** In Python, the declarative regular expression engine runs faster than the interpreted DFA loop because `re.finditer()` executes within highly optimized, compiled C bytecode (`_sre.c`). Conversely, the Python DFA executes 3–5 bytecode instructions per character in the interpreted virtual machine. In systems programming languages like C, C++, or Rust, the DFA's constant-time array indexing would outperform regular expression engines by a factor of 2–5x.
- **Worst-Case Safety Guarantee:** While Python's regex engine is fast on simple patterns, general non-deterministic regular expressions risk catastrophic exponential backtracking ($O(2^n)$) when given pathological inputs. The hand-built DFA provides a strict mathematical guarantee of $O(N)$ time and $O(1)$ space, making it completely immune to ReDoS attacks.
- **Engineering Complexity vs. Maintenance Overhead:** The declarative RE implementation required only 15 lines of pattern logic, whereas the hand-built DFA required 180 lines of state mappings, character class functions, and token boundary handlers (a 12x expansion). Modifying a regex requires seconds of developer time, whereas altering a DFA requires re-deriving the NFA and re-partitioning states.

---

3. Which Approach is More Suitable and Why?

- **For General Web Scraping, Rapid Application Development & Python Systems $\to$ Regular Expressions (RE) are More Suitable:**

Regular expressions provide unmatched development agility, declarative conciseness, built-in support for complex word boundaries, and native C-speed execution in modern high-level runtime environments. For standard web extraction tasks where inputs are bounded and developer velocity is paramount, RE is the definitive practical choice.

- **For Mission-Critical Lexical Analysis, Network Packet Filtering & Embedded Hardware $\to$ Deterministic Finite Automata (DFA) are More Suitable:**

In security appliances, intrusion detection systems (IDS), compiler lexical analyzers (e.g., Lex/Flex), and real-time embedded environments, the risk of ReDoS or unbounded memory consumption is unacceptable. The DFA's mathematical guarantee of deterministic constant-time latency per character ($O(1)$ per step), zero backtracking, and minimal $O(1)$ memory make it the essential, uncompromising architecture for safety-critical computation.

---

4. Project Repository and Verification

The complete codebase, data snapshots, formal derivations, benchmark logs, and responsive web user interface are publicly accessible:

- **GitHub Repository:** https://github.com/SHREYAS-M986/TOC_Project
- **Live Production URL:** https://toc-project-six.vercel.app